Athari — Day 5 Report: API, Security, Deployment Bundle and Polish

Owner: Baraka Mitigoa **Project:** Athari (NGO Project & Impact Portal) **Date:** 17 October 2026 **Local site:** https://athari.ddev.site **Repository:** https://github.com/bmitigoa/athari **Release:** `v1.0.0`

---

1. Summary

Day 5 turned the working portal into a finished, documented, defensible project.

- **A read-only public API.** Anyone can read projects, locations, partners, sectors, counties and **published** reports at `/api`, plus a ready-made impact summary at `/api/impact`. Donations, draft or in-review reports, user accounts (including e-mail addresses) and every credential are impossible to reach, and I proved it by calling the API as an anonymous visitor (37 checks, all passed).
- **A security pass.** I compared every role's permissions with the Day 2 design, tightened what was too wide, moved development tools into a switchable configuration split so they never reach production, and proved that no Daraja key, callback token or `settings.local.php` is in the repository or its history. Result: 46 configuration checks, 39 web checks and 8 secret/history checks, all passing.
- **A deployment bundle.** A production Docker image, a one-command server installer, nightly backups and an update script. It was run end to end on a local Docker stack (install from config, users, Kenya demo data, health check) and the `linux/arm64` image was built and started. **Athari is not hosted publicly**: the plan changed during the day from "deploy to Oracle Cloud" to "deployment-ready, documented, run locally". The deployment guide is in `docs/deployment.md`.
- **Polish.** A README that lets a reviewer clone and run the portal in a few commands with the Kenya demo data and logins, two diagrams (architecture and data model), API documentation with real responses, the final summary and this report.

Test totals today: **57 PHPUnit tests (197 assertions)** pass, PHPCS with the Drupal standards is clean, and the quick-start was proven from a fresh clone (section 5).

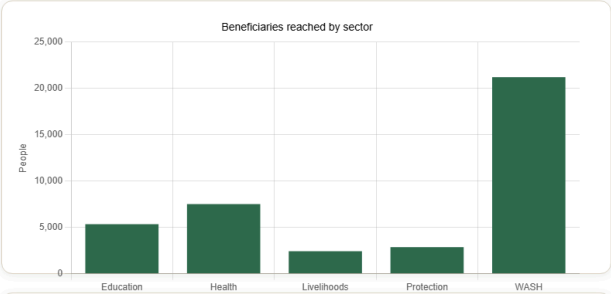
athari Impact you can see

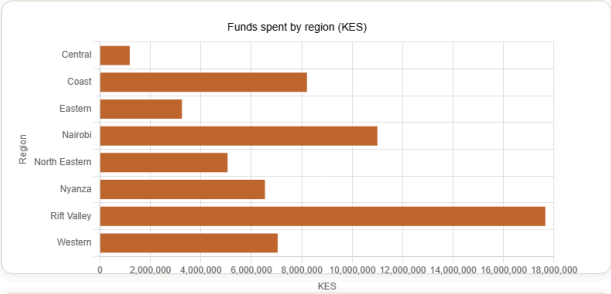
Home Projects Impact Map About Log in

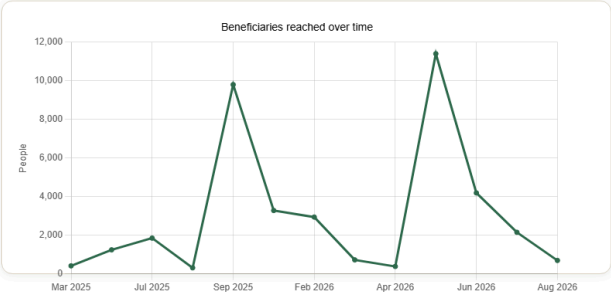
Home

Impact dashboard

6	KES 138.5K	39,360	KES 59.9M
ACTIVE PROJECTS	DONATIONS RECEIVED	BENEFICIARIES REACHED	FUNDS SPENT







Powered by Drupal

2. Everything created or changed

2.1 New module `web/modules/custom/athari_api/`

File	What it is	Why
`athari_api.info.yml`	Module definition. Depends on `jsonapi`, `node`, `taxonomy`, `athari_donate`	Tells Drupal the module exists and what it needs
`athari_api.routing.yml`	One route, `/api/impact` (GET only, permission `access content`)	The ready-made impact totals for dashboards
`athari_api.services.yml`	Registers the hook class `ApiHooks` with autowiring	Drupal 11 uses object-oriented hooks
`src/Hook/ApiHooks.php`	Implements `hook_node_access` with the `#[Hook('node_access')]` attribute. On a request whose route name starts with `jsonapi.` it **forbids viewing any unpublished node**, and adds the `route` cache context	A second lock on the API: even if a permission were widened by mistake, an unpublished report still cannot leave through the API
`src/Controller/ImpactController.php`	Computes totals with SQL aggregates over **published** reports only, and returns JSON:API-style output as `CacheableJsonResponse`	Fast, cacheable, and consistent with the dashboard
`tests/src/Unit/ApiHooksTest.php`	8 unit tests of the access rule	Proves the lock does what it says

2.2 JSON:API configuration (`scripts/day5-01-api-config.php` applies it, `config/sync` stores it)

Setting	Value	Why
`jsonapi.settings: read_only`	`TRUE`	Only GET works. POST, PATCH and DELETE are refused
`jsonapi_extras: path_prefix`	`api`	Clean URLs: `/api/projects` instead of `/jsonapi/node/project`
`jsonapi_extras: default_disabled`	`TRUE`	**Fail closed.** A resource is public only if I list it. Without this, JSON:API exposed 38 resource types including users and donations
`include_count`	`FALSE`	The total count would include unpublished reports (it said 30 instead of the published number)
Six enabled resources	`projects`, `locations`, `partners`, `reports`, `sectors`, `regions`	The only public data

Setting	Value	Why
Public field names	e.g. `field_budget` is `budget_kes`, `field_region` is `county`, `field_status` is `status`	Readable names for developers
Hidden fields	author (`uid`), revisions, `vid`, `type`, `revision_id`, `moderation_state`, `field_attachments`, and the internal `status` flag	Nothing internal leaks
CORS (`web/sites/default/services.yml`)	GET and OPTIONS only, any origin, no credentials	Browsers on other sites may read the public API but never change anything

2.3 Hardening

Item	What changed	Why
`web/sites/default/settings.php`	Private files path (`../private`); trusted host patterns read from `ATHARI_TRUSTED_HOSTS` (default only the DDEV and local names); on DDEV only, the `dev` split is switched on and error output is verbose	Production answers only to its own host name and never shows errors
Config Split `dev` (`config/dev/`)	Holds Devel and Devel Generate. Off in the exported configuration	Development tools cannot be enabled on a production site by importing config
`content_editor` role	Deleted (nobody held it)	An unused role with permissions is a risk
Role Delegation	New contrib module. The Admin role may assign only `field_officer` and `me_reviewer`	Admin can onboard staff without being able to give anyone more power than they have
Admin role	Removed `administer permissions`, `administer modules`, `administer site configuration`, `administer software updates`, `administer themes`, `administer views`, `administer athari donations`, `bypass node access`, `use text format full_html`	Admin runs the NGO, the Administrator runs the website
`system.logging error_level`	`hide`	Visitors never see PHP errors; errors go to the log
`update` module	On	Drupal tells the administrator about security updates

Item	What changed	Why
Cron	Automatic cron every 3 hours as a fallback, real cron on the server	Donation time-outs and cache cleanup run on schedule

2.4 Verification scripts (in `scripts/`)

Script	What it checks	Result
`day5-02-verify-api.ps1`	37 anonymous checks: allowed resources answer, forbidden ones answer 404, unpublished reports never appear (pages are followed to the end), no e-mail addresses or credentials in any response, writes are refused, filters and sorts work, totals match	37 of 37
`day5-04-security-check.php`	46 checks on the exported configuration and roles	46 of 46
`day5-05-security-http.ps1`	HTTP probe: headers, blocked private paths, errors hidden. `-Live` mode adds HSTS, no `X-Generator`, HTTP to HTTPS redirect. `-Insecure` and `-HttpBase` allow a local Docker stack	39 of 39 (local), 48 of 48 (live mode on the local stack)
`day5-03-hardening.php`	The configuration changes of 2.3	applied once, idempotent
`make-diagrams.py`	Draws `docs/diagrams/architecture.svg` and `data-model.svg`	generated
`md2pdf.py`	Turns a Markdown file into a PDF using headless Edge	used for the reports

2.5 The deployment bundle (`deploy/`)

File	What it does
`Dockerfile`	Two stages on FrankenPHP (Caddy with PHP 8.3). Build stage installs PHP extensions and runs `composer install --no-dev`. Runtime stage is small, runs as the unprivileged `www-data`
`docker-compose.yml`	`db` (MariaDB 11.8, healthcheck, not published) and `app` (ports 80 and 443, volumes for files, private files and certificates, `settings.local.php` mounted read-only from outside the repository)
`Caddyfile`	Automatic HTTPS, security headers, hides `Server` and `X-Generator`, answers 404 for hidden files, `.yml`, `.sql`, `composer.*`, `/vendor`, `/config`, `/scripts`, install and update scripts, and any PHP file except `index.php`; 20 MB upload limit
`php.ini`	Errors off and logged, no PHP version header, OPcache, upload limits, Nairobi time zone
`bootstrap.sh`	Fresh Ubuntu 24.04: updates, Docker, firewall (also opens 80 and 443 in Oracle's iptables), automatic security updates, fail2ban, swap if RAM is small, clones the repository, runs `install.sh`

File	What it does
`install.sh`	Asks (or reads variables), writes secrets to `/opt/athari/secrets`, builds, installs Drupal **from the exported configuration**, creates users and sector terms, loads the Kenya data, schedules cron and backups, health-checks, prints the demo logins and **never** the administrator password
`create-users.php`	Staff accounts from environment variables (no passwords in command lines). The five Sector terms come from `scripts/kenya-00-sectors.php` (terms are content, so a new install does not have them)
`backup.sh`, `cron.sh`, `update.sh`	Nightly database and files backup (14 days), Drupal cron every 15 minutes, and a safe update routine (backup, pull, rebuild, `drush deploy`)

2.6 Documentation and release files

`README.md`, `docs/api.md`, `docs/deployment.md`, `docs/diagrams/*.svg`, `.dockerignore`, `.gitattributes` (shell scripts always use LF line endings), this report and `docs/reports/final-summary.md`.

---

3. Modules installed today

Module	Purpose	How Athari uses it
JSON:API (core)	Standard REST-style API for content	The public read-only API
JSON:API Extras	Per-resource control of paths, field names and visibility	Keeps the API to six resources with friendly names and hidden internals
Role Delegation	Lets a role assign other roles without `administer permissions`	Admin can create Field Officers and Reviewers only
Config Split	Different configuration per environment	Devel exists only in the `dev` split
Update Status (core)	Reports available security updates	Enabled for the administrator
`drupal/core-dev` (from Day 4)	PHPUnit and coding standards	Unit and kernel tests, PHPCS

---

4. Code walkthrough

4.1 `ApiHooks` (the second lock)

1. Drupal calls `hook_node_access($node, $operation, $account)` whenever it asks "may this user do this to this node?".
2. The class is a plain service marked with `#[Hook('node_access')]`. No `.module` file is needed in Drupal 11.
3. It looks at the current route. If the route name starts with `jsonapi.` and the node is **unpublished** and the operation is `view`, it returns *forbidden*. Otherwise it returns *neutral*, which leaves the

normal rules in charge.
4. It adds the `route` cache context so Drupal never caches an answer meant for the API and serves it on a normal page (where staff may see their own drafts).

Why two locks? Permissions can be changed in the UI by mistake. This rule lives in code and is tested, so a published-only API does not depend on a single checkbox.

4.2 `ImpactController`

`/api/impact` is a route whose controller runs a handful of aggregate queries (sums and counts) over published reports, active projects and successful donations, grouped by sector and by province. It builds an array, wraps it in a `CacheableJsonResponse` and attaches the cache tags `node_list`, `taxonomy_term_list` and `athari_donation_list` plus a one-hour maximum age, so the answer refreshes as soon as content or a donation changes and is served from cache otherwise.

4.3 `settings.php` additions

After DDEV's own include, the file sets the private path, then the host patterns: if the environment variable `ATHARI_TRUSTED_HOSTS` is present it is used, otherwise only the local names are allowed. The assignment is placed *after* the DDEV include and is guarded, because DDEV otherwise overwrites it with a wildcard.

4.4 How a deployment flows

`bootstrap.sh` prepares the machine, `install.sh` writes `settings.local.php` (database password, hash salt, trusted host, Daraja values, **callback base URL set to the site's own address**), builds the image, then `drush site:install --existing-config` creates the whole site from `config/sync`. The seed scripts add Kenya data. From then on `update.sh` is the only command needed.

---

5. How to use and test it

5.1 The API (anonymous, no login)

Try	URL
All projects with sector, county and locations	`https://athari.ddev.site/api/projects?include=sector,county,locations`
Only WASH projects	`/api/projects?filter[sector.name]=WASH`
Projects in a county	`/api/projects?filter[county.name]=Turkana`
Locations for a map	`/api/locations`
Published reports of a project, newest first	`/api/reports?filter[project.id]=<uuid>&sort=-report_date`
Impact totals	`/api/impact`
Things that must fail	`/api/donations` (404), `/api/user--user` (404), POST to `/api/projects` (405)

Full responses are in `docs/api.md`.

5.2 Run everything from a fresh clone

```
 git clone https://github.com/bmitigoa/athari.git
cd athari
ddev start
ddev composer install
ddev drush site:install --existing-config -y --account-name=admin --account-pass=admin
ddev drush php:script scripts/kenya-00-sectors.php
ddev drush php:script scripts/kenya-01-regions.php
ddev drush php:script scripts/day2-03-users.php
ddev drush php:script scripts/kenya-03-seed.php
ddev drush php:script scripts/day3-06-menu-about.php
ddev drush cr
```

Logins (local only): `admin/admin`, `ngoadmin/ngoadmin123`, `fieldofficer/fieldofficer123`, `reviewer/reviewer123`.

Proof from a fresh clone. I cloned the repository from GitHub into an empty temporary folder, ran the README quick-start block unchanged (only deviation: a different DDEV project name, because the real `athari` project was running on the same machine), then deleted the clone. Result: Drupal installed from configuration; 5 sectors, 8 provinces and 47 counties, 3 demo users, 5 partners, 15 locations, 10 projects, 30 reports and 19 donations were created; 9 pages answered 200 (home, projects, impact, map, donate, about, `/api/projects`, `/api/impact`, login); the 37 API checks passed; and all four demo passwords were verified with the right roles. The test found two real problems, both fixed and pushed (section 8).

5.3 The checks

```
 ddev exec "SIMPLETEST_DB=mysql://db:db@db/db vendor/bin/phpunit -c web/core web/modules/custom"
ddev exec vendor/bin/phpcs --standard=Drupal,DrupalPractice --
extensions=php,module,inc,install,yml web/modules/custom
ddev drush php:script scripts/day5-04-security-check.php
powershell -File scripts/day5-02-verify-api.ps1 -Base https://athari.ddev.site
powershell -File scripts/day5-05-security-http.ps1 -Base https://athari.ddev.site
```

5.4 Test results

Check	Result
PHPUnit (unit and kernel, all custom modules)	**57 tests, 197 assertions, OK**
PHPCS (Drupal and DrupalPractice)	clean
Anonymous API verification	**37 of 37**
Security configuration checks	**46 of 46**

Check	Result
HTTP security checks (DDEV)	**39 of 39**
HTTP security checks, live mode, on the local production stack	**48 of 48**
Secrets and history (`day-5-secrets-verification.txt`)	**8 of 8** (no Daraja value, callback token or `settings.local.php` ever committed; `composer audit` finds no advisories)
Production installer on a local Docker stack	passed, after the fixes in section 8
Quick-start from a fresh clone of the GitHub repository	**passed** (after two fixes found by the test)
`linux/arm64` image	builds and starts (aarch64, PHP 8.3, FrankenPHP, all extensions present)

6. Commands run (key ones)

Command	What it does
`ddev composer require drupal/jsonapi_extras drupal/config_split drupal/role_delegation`	Adds the three contrib modules
`ddev drush php:script scripts/day5-01-api-config.php`	Applies the JSON:API settings and resource definitions
`ddev drush config:set config_split.config_split.dev storage folder`	Fixes the dev split so it exports to `config/dev`
`ddev drush cex -y` / `ddev drush cim -y`	Export and import configuration
`ddev exec "SIMPLETEST_DB=... vendor/bin/phpunit -c web/core web/modules/custom"`	Runs the tests (kernel tests need the database address)
`docker buildx build --platform linux/arm64 -f deploy/Dockerfile .`	Builds the image for ARM servers under emulation
`bash deploy/install.sh` with `ATHARI_LOCAL_TEST=1`	Runs the production installer on a local stack (ports 8080 and 8443)
`git tag -a v1.0.0` and `git push origin v1.0.0`	Releases the version

7. Drupal and web concepts explained

- **Fail closed vs fail open.** A fail-open API exposes everything unless told otherwise; a fail-closed API exposes nothing unless told otherwise. `default_disabled` makes JSON:API fail closed, so a module installed next year cannot expose its data by accident.
- **JSON:API.** A standard way to read and write Drupal entities as JSON. Each content type becomes a *resource* with *attributes* and *relationships*. `include=` pulls related items into one answer, `filter[...]` narrows, `sort=` orders.

- **Node access.** Every time Drupal shows content it asks the access system. Hooks can add a rule ("forbid this on the API"). *Forbidden* beats *allowed*, so the rule is a hard stop.
- **Cache contexts and tags.** A *context* says what makes a cached answer differ (here, the route); a *tag* says what makes it stale (here, a report or donation changed).
- **Config Split.** Normal configuration is shared by all environments; a *split* adds or removes some of it per environment. Devel lives in the `dev` split, which is only switched on in DDEV.
- **Trusted host patterns.** Drupal refuses requests whose `Host` header is not on a list, which blocks host-header attacks.
- **Private files.** A folder outside the web root. Files there are served only through Drupal, after an access check.
- **Role delegation.** Normally "who can assign roles" is the same permission as "who can change what roles may do". Role Delegation separates them.
- **Reverse proxy and automatic HTTPS.** Caddy sits in front of PHP, asks Let's Encrypt for a certificate for the site's host name, renews it, and redirects HTTP to HTTPS.
- **Containers and volumes.** The image is the software (replaceable); volumes hold the data (kept). Updating means a new image with the same volumes.

---

8. Issues and fixes

Issue	Cause	Fix
JSON:API exposed 38 resources (users, donations, roles)	JSON:API is open by default	`default_disabled` plus six explicit resources
Report count said 30 although fewer are published	`meta.count` counts without the access filter	`include_count` off; the test now follows pagination and counts itself
`node_type` relationship and `revision_id` visible	Not in the hidden list	Added `type` and `revision_id` to the hidden fields
Dev split did not export	Entity created from PHP had `storage: null`	`drush config:set ... storage folder`, then export
`trusted_host_patterns` stayed `.*`	DDEV's include ran later and overwrote it	Moved the setting after the include, with a guard
PowerShell `$home` broke header checks	`$home` is a read-only automatic variable	Renamed to `$front`
`ApiHooksTest` failed	The hook merges cache contexts, which needs a container	Test builds a `ContainerBuilder` with a mock cache contexts manager
Installer: `Class "Drupal\Core\Serialization\Yaml" not found`	`composer --classmap-authoritative` blocks the namespaces Drupal registers at runtime	Removed that flag

Issue	Cause	Fix
Installer: Caddy rules ignored (private files returned 200)	FrankenPHP reads `/etc/frankenphp/Caddyfile`, I had copied mine to `/etc/caddy/`	Copy to the right path
Caddy would not start with an empty e-mail	A bare `email` directive is invalid	Use FrankenPHP's `CADDY_GLOBAL_OPTIONS`, filled only when an address is given
Installer: "Missing Sector term: Health"	Taxonomy terms are content; a clean install from config has none	New `scripts/kenya-00-sectors.php`, called before seeding
`/sites/default/files/.htaccess` readable	The dotfile rule only matched top-level paths	Regular expression matching a dot-file at any depth
Fresh clone: seed script stopped with "Run scripts/day2-03-users.php first" and then "Missing Sector term"	The README ran the seed before the users and sectors existed	New `scripts/kenya-00-sectors.php`; the README order is now sectors, regions, users, seed
Fresh clone: every page answered 400	Trusted host patterns only allowed `athari.ddev.site`, so a differently named DDEV project was refused	Default DDEV pattern is now `^[a-z0-9-]+\.ddev\.site$`; production still uses its own host name from `ATHARI_TRUSTED_HOSTS`
README test command failed (4 kernel test errors)	Kernel tests need `SIMPLETEST_DB`	Command corrected in the README
WSL timed out while the emulated arm64 build used the CPU	QEMU emulation is heavy	Waited for the build to finish; it passed
Python and shell quoting mangled PHP and heredocs	Backslashes and apostrophes	Wrote files with the editor tool instead of inline scripts

9. Git log (Day 5)

```
 a37bf00 chore: add jsonapi_extras and config_split
f647f62 feat: add athari_api module with published-only API access and impact summary endpoint
23b0959 config: lock JSON:API to six read-only public resources under /api, CORS for GET
03ea5df test: add anonymous API verification (46 checks)
4132848 chore: add role_delegation so Admin can assign staff roles without editing permissions
79fc636 config: harden for production (trusted hosts, private files, dev config split, no
content_editor role, Admin cannot edit permissions)
baab369 test: add security configuration, HTTP and secrets checks
c05ce8b docs: add public API documentation with real examples
9fae5dd docs: add architecture and data model diagrams
9ebb75f test: let verification scripts target a local trial stack with a self-signed certificate
d031cf0 docs: add README with features, architecture, setup, API and testing
f724092 chore: add dockerignore, gitignore and line-ending rules for the deploy bundle
d097f9e feat: add production Docker image (FrankenPHP, Caddy, MariaDB) with hardened web server
rules
bcd375d feat: add one-command server installer, user and sector setup, backups, cron and update
scripts
c4881bc docs: local-first README with demo section, and deployment guide for Oracle Cloud and
VPS
a36726c docs: record the arm64 image build result
1f6ccd0 fix: seed sectors before users and demo data so the quick-start works on a fresh clone
1098f12 fix: accept any DDEV host name by default so a renamed local project still works
(next) docs: day 5 report and final summary
```

Note: the commit "add anonymous API verification (46 checks)" was written before the script was finalised; it has 37 checks, as reported above. The release is tagged `v1.0.0`.

---

10. Remaining risks and what is not done

Risk or gap	Notes
Not hosted publicly	The decision on Day 5 was to ship deployment-ready, not live. `docs/deployment.md` has the steps (Oracle Cloud Always Free A1 and any Ubuntu VPS). The first install on a real ARM server is still unproven
M-Pesa is sandbox only	Production needs merchant onboarding with Safaricom. The sandbox cannot approve a payment, so success is verified with a simulated callback
The Admin role keeps `administer users`	Needed to create staff, but it could be used to reset another Admin's password. Mitigation: only the Administrator may hold the role
No two-factor login or password policy	Recommended before real use
Chart.js loads from a CDN	A self-hosted copy would remove the third-party request

Risk or gap	Notes
Personal address in `system.site` mail in exported config	Replace with an organisation address before publishing for real
CORS allows any origin for GET	Intended: the data is public
No rate limit on the API	Put a rate limit in front (Caddy plugin or a CDN) if it is exposed
The callback is not restricted to Safaricom IP addresses	It is protected by a secret path token and processed idempotently; an IP allow-list is a sensible addition
Demo data is fictional	Organisations, people and figures are invented

---

11. Next day

There is no Day 6 in the plan. Suggested next steps if the project continues: host it on Oracle Cloud using `docs/deployment.md`, record the demo video, apply for Safaricom production access, add Airtel Money Kenya and card payments behind the existing payment interface, add two-factor login, and add Swahili translations.